

```
In [1]: import numpy as np
```

```
In [2]: np.__version__
```

```
Out[2]: '2.1.3'
```

```
In [74]: import sys  
sys.version
```

```
Out[74]: '3.13.5 | packaged by Anaconda, Inc. | (main, Jun 12 2025, 16:37:03) [MSC v.1929 64 bit (AMD64)]'
```

```
In [ ]: #List creation
```

```
In [3]: mylist = [0,1,2,3,4,5]  
mylist
```

```
Out[3]: [0, 1, 2, 3, 4, 5]
```

```
In [4]: type(mylist)
```

```
Out[4]: list
```

```
In [ ]: #Convert List to array
```

```
In [6]: arr = np.array(mylist)  
arr
```

```
Out[6]: array([0, 1, 2, 3, 4, 5])
```

```
In [7]: type(arr)
```

```
Out[7]: numpy.ndarray
```

```
In [ ]: #The above array is 1D array
```

```
In [ ]: #Important functions in numpy
```

```
In [9]: pip install numpy
```

Requirement already satisfied: numpy in c:\users\sid balurkar\anaconda3\lib\site-packages (2.1.3)
Note: you may need to restart the kernel to use updated packages.

```
In [11]: np.arange(10)      #with int
```

```
Out[11]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [12]: np.arange(20)
```

```
Out[12]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,  
               17, 18, 19])
```

```
In [75]: np.arange(15)
```

```
Out[75]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
```

```
In [14]: np.arange(5.0)     #with float
```

```
Out[14]: array([0., 1., 2., 3., 4.])
```

```
In [77]: np.arange(3.0)
```

```
Out[77]: array([0., 1., 2.])
```

```
In [15]: np.arange(0,5)
```

```
Out[15]: array([0, 1, 2, 3, 4])
```

```
In [16]: np.arange(10,20)   #range
```

```
Out[16]: array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

```
In [78]: np.arange(0,5)
```

```
Out[78]: array([0, 1, 2, 3, 4])
```

```
In [17]: np.arange(20,10)    #1st arg < 2nd arg
```

```
Out[17]: array([], dtype=int64)
```

```
In [18]: np.arange(-20,10)
```

```
Out[18]: array([-20, -19, -18, -17, -16, -15, -14, -13, -12, -11, -10, -9, -8,
               -7, -6, -5, -4, -3, -2, -1,  0,  1,  2,  3,  4,  5,
                6,  7,  8,  9])
```

```
In [19]: np.arange(-16,10)
```

```
Out[19]: array([-16, -15, -14, -13, -12, -11, -10, -9, -8, -7, -6, -5, -4,
               -3, -2, -1,  0,  1,  2,  3,  4,  5,  6,  7,  8,  9])
```

```
In [22]: ar = np.arange(-30,20)      #Creating variable
```

```
In [23]: ar
```

```
Out[23]: array([-30, -29, -28, -27, -26, -25, -24, -23, -22, -21, -20, -19, -18,
               -17, -16, -15, -14, -13, -12, -11, -10, -9, -8, -7, -6, -5,
               -4, -3, -2, -1,  0,  1,  2,  3,  4,  5,  6,  7,  8,
                9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

```
In [ ]: #The above is multi dimensional
```

```
In [79]: np.arange(10,10)
```

```
Out[79]: array([], dtype=int64)
```

```
In [24]: #Array with no parameters
np.arange()
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[24], line 2
      1 #Array with no parameters
----> 2 np.arange()

TypeError: arange() requires stop to be specified.
```

```
In [ ]: #arange() requires min one parameter
```

```
In [25]: np.arange(10,30,5)    #step count 5
```

```
Out[25]: array([10, 15, 20, 25])
```

```
In [27]: np.arange(0,10,3)
```

```
Out[27]: array([0, 3, 6, 9])
```

```
In [28]: np.arange(10,30,5,8)
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[28], line 1  
----> 1 np.arange(10,30,5,8)  
  
TypeError: Cannot interpret '8' as a data type
```

```
In [ ]: #arange() takes max 3 arguments and min 1 argument
```

```
In [29]: np.zeros(10)    #parameter tuning
```

```
Out[29]: array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
In [31]: np.zeros(3)
```

```
Out[31]: array([0., 0., 0.])
```

```
In [ ]: #The above creates float values
```

```
In [32]: np.zeros(3, dtype=int)    #1D array with 0 values - Hyperparameter tuning
```

```
Out[32]: array([0, 0, 0])
```

```
In [ ]: #2D array
```

```
In [34]: np.zeros((2,2), dtype=int)
```

```
Out[34]: array([[0, 0],  
                [0, 0]])
```

```
In [35]: zero = np.zeros([2,2])
print(zero)
print('####')
print(type(zero))
```

```
[[0. 0.]
 [0. 0.]]
####
<class 'numpy.ndarray'>
```

```
In [37]: np.zeros((2,10))      #2 by 10 matrix
```

```
Out[37]: array([[0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
```

```
In [81]: np.zeros((10,30))
```

```
Out[81]: array([[0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
```

```
In [84]: n1 = (6,8)
print(np.zeros(n1))
```

```
[[0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]]
```

```
In [85]: np.zeros((10,10), dtype=int)    #10 rows by 10 columns
```

```
Out[85]: array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]])
```

```
In [ ]: #1D value with 1 value float
```

```
In [39]: np.ones(3)
```

```
Out[39]: array([1., 1., 1.])
```

```
In [40]: np.ones((3), dtype=int)
```

```
Out[40]: array([1, 1, 1])
```

```
In [41]: np.ones((3,3), dtype=int)
```

```
Out[41]: array([[1, 1, 1],
                [1, 1, 1],
                [1, 1, 1]])
```

```
In [86]: np.ones((5,3), dtype = int)
```

```
Out[86]: array([[1, 1, 1],
               [1, 1, 1],
               [1, 1, 1],
               [1, 1, 1],
               [1, 1, 1]])
```

```
In [ ]: #3Darray with ones
```

```
In [42]: np.two((2,3))    #not an inbuilt function
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[42], line 1
----> 1 np.two((2,3))

File ~\anaconda3\Lib\site-packages\numpy\__init__.py:414, in __getattr__(attr)
    411     import numpy.char as char
    412     return char.chararray
--> 414 raise AttributeError("module {!r} has no attribute "
    415                        "{!r}".format(__name__, attr))

AttributeError: module 'numpy' has no attribute 'two'
```

```
In [ ]: #How otp is generated
```

```
In [43]: rand(3,2)    #module not given
```

```
-----
NameError                                    Traceback (most recent call last)
Cell In[43], line 1
----> 1 rand(3,2)

NameError: name 'rand' is not defined
```

```
In [44]: random.rand(3,2)    #package is missing
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[44], line 1  
----> 1 random.rand(3,2)  
  
NameError: name 'random' is not defined
```

```
In [45]: np.random.rand(3,2)      #np is package, random is module, rand is function
```

```
Out[45]: array([[0.32486114, 0.14573522],  
               [0.78006674, 0.15249568],  
               [0.0572085 , 0.85412526]])
```

```
In [46]: np.random.rand(3,2)
```

```
Out[46]: array([[0.19855582, 0.9296781 ],  
               [0.57240105, 0.64526484],  
               [0.5851315 , 0.23899394]])
```

```
In [47]: np.random.rand(3,2)
```

```
Out[47]: array([[0.72918362, 0.28978913],  
               [0.94076744, 0.76261188],  
               [0.97994983, 0.49717958]])
```

```
In [48]: np.random.rand(5)
```

```
Out[48]: array([0.87846403, 0.73481347, 0.97556067, 0.75093515, 0.49163728])
```

```
In [50]: np.random.rand(3,5)      #3 rows and 5 columns
```

```
Out[50]: array([[0.54184606, 0.215046 , 0.87894036, 0.86847579, 0.46739398],  
               [0.59159868, 0.63724393, 0.46980582, 0.90531276, 0.33570043],  
               [0.64531796, 0.48652954, 0.25976754, 0.98038802, 0.72916455]])
```

```
In [ ]: #otp is not decimal. Hence, use randint()
```

```
In [51]: np.random.randint(4,6)    #print either 4 or 5. Low limit to 'n-1' high limit
```

```
Out[51]: 4
```



```
In [52]: np.random.randint(4,6)
```

```
Out[52]: 5
```

```
In [87]: np.random.randint(0,1)
```

```
Out[87]: 0
```

```
In [88]: np.random.randint(1)
```

```
Out[88]: 0
```

```
In [55]: np.random.randint(2,20,4)      #Generate 4 random numbers between 2 to 19
```

```
Out[55]: array([ 2,  3,  3, 15], dtype=int32)
```

```
In [56]: np.random.randint(0,9,4)      #2nd parameter is exclusive it is n-1 so 0 to 8. give 4 random numbers
```

```
Out[56]: array([3, 0, 4, 5], dtype=int32)
```

```
In [57]: np.random.randint(30,20,10)    #1st arg < 2nd arg only
```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[57], line 1  
----> 1 np.random.randint(30,20,10)  
  
File numpy\random\mttrand.pyx:796, in numpy.random.mtrand.RandomState.randint()  
  
File numpy\random\_bounded_integers.pyx:1425, in numpy.random._bounded_integers._rand_int32()  
  
ValueError: low >= high
```

```
In [58]: np.random.randint(-30,20,10)
```

```
Out[58]: array([-8, -15, -15, -25, -15, -9,  8, -2, -20,  3], dtype=int32)
```

```
In [59]: np.random.randint(30,20,10,5)    #can not take 4 arguments
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[59], line 1
----> 1 np.random.randint(30,20,10,5)

File numpy\random\mtrand.pyx:777, in numpy.random.mtrand.RandomState.randint()

TypeError: Cannot interpret '5' as a data type
```

```
In [60]: np.random.randint(10,40,(10,10))       #(10 by 10 matrix) with 10 - 39 values
```

```
Out[60]: array([[33, 11, 28, 10, 12, 10, 36, 36, 16, 23],
               [11, 20, 37, 28, 34, 28, 31, 24, 34, 10],
               [39, 35, 38, 13, 13, 36, 26, 11, 24, 14],
               [27, 33, 21, 14, 19, 27, 13, 28, 11, 33],
               [20, 10, 22, 31, 14, 10, 38, 39, 13, 29],
               [11, 12, 18, 18, 30, 14, 23, 37, 22, 18],
               [18, 19, 20, 14, 31, 11, 35, 32, 24, 20],
               [21, 12, 33, 27, 30, 10, 24, 21, 18, 35],
               [39, 37, 12, 22, 28, 32, 25, 25, 13, 29],
               [37, 22, 14, 13, 13, 15, 19, 38, 33, 31]], dtype=int32)
```

```
In [61]: np.random.randint(10,40,(10,10))
```

```
Out[61]: array([[35, 27, 25, 10, 18, 15, 29, 38, 35, 12],
               [11, 16, 34, 28, 11, 12, 24, 10, 13, 26],
               [20, 18, 33, 35, 35, 27, 21, 10, 36, 32],
               [28, 23, 16, 14, 14, 11, 32, 39, 39, 24],
               [33, 11, 10, 24, 31, 30, 35, 32, 30, 33],
               [20, 17, 30, 36, 34, 24, 20, 28, 11, 16],
               [26, 38, 38, 26, 38, 32, 16, 24, 38, 29],
               [21, 12, 36, 38, 36, 37, 28, 16, 25, 17],
               [14, 18, 31, 32, 17, 29, 21, 29, 13, 28],
               [15, 37, 34, 17, 34, 30, 20, 20, 31, 10]], dtype=int32)
```

```
In [62]: np.random.randint(1,100,(12,12))
```

```
Out[62]: array([[93, 34, 10, 95,  4, 75, 58, 87, 78, 71, 71, 95],
               [78, 51, 44, 48,  2, 47, 46, 86, 60, 45, 40, 78],
               [92, 83,  9, 57, 42, 64, 35, 53,  7, 16, 84, 20],
               [64, 46,  5, 14, 19, 15, 19, 61, 12, 58, 65, 96],
               [ 4, 75, 66, 87, 61, 31,  4, 90, 11, 51, 11, 55],
               [19, 57, 90, 16, 82, 38, 83, 21, 51, 57, 95, 10],
               [75, 80, 92, 71, 24, 20, 67, 63, 85, 79, 54, 88],
               [57, 16, 68, 31, 74, 73, 72, 49, 97, 23, 33, 64],
               [98, 60, 84, 43, 27, 46,  1, 81, 87,  5, 80, 89],
               [42,  1, 25, 64, 21, 93, 91, 29,  1,  4, 46, 97],
               [87, 36, 17,  7, 33, 68, 43, 58,  9, 64, 84, 12],
               [77, 84, 72, 13,  6, 33,  6, 26,  8, 53, 39, 28]], dtype=int32)
```

```
In [63]: np.arange(1,13)
```

```
Out[63]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
```

```
In [64]: np.arange(1,13).reshape(3,4)    #3 rows 4 columns are arranged in proper order
```

```
Out[64]: array([[ 1,  2,  3,  4],
               [ 5,  6,  7,  8],
               [ 9, 10, 11, 12]])
```

```
In [65]: np.arange(1,13).reshape(3,5)    #3 multiply by 5 is 15, is incorrect. in reshape() the number should match
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[65], line 1
----> 1 np.arange(1,13).reshape(3,5)

ValueError: cannot reshape array of size 12 into shape (3,5)
```

```
In [66]: np.arange(1,13).reshape(1,16)
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[66], line 1
----> 1 np.arange(1,13).reshape(1,16)

ValueError: cannot reshape array of size 12 into shape (1,16)
```

```
In [69]: np.arange(1,13).reshape(1,12)
```

```
Out[69]: array([[ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12]])
```

```
In [70]: np.arange(1,13).reshape(6,2)
```

```
Out[70]: array([[ 1,  2],
               [ 3,  4],
               [ 5,  6],
               [ 7,  8],
               [ 9, 10],
               [11, 12]])
```

```
In [89]: np.arange(1,13).reshape(12,1)
```

```
Out[89]: array([[ 1],
               [ 2],
               [ 3],
               [ 4],
               [ 5],
               [ 6],
               [ 7],
               [ 8],
               [ 9],
               [10],
               [11],
               [12]])
```

```
In [ ]: #Slicing in matrix
```

```
In [71]: b = np.random.randint(10,20,(5,4))
b
```

```
Out[71]: array([[10, 12, 13, 19],
               [16, 10, 14, 15],
               [14, 14, 14, 14],
               [16, 12, 13, 11],
               [10, 18, 14, 14]], dtype=int32)
```

```
In [72]: type(b)
```

Out[72]: numpy.ndarray

In [73]: `b[:]`

Out[73]: array([[10, 12, 13, 19],
 [16, 10, 14, 15],
 [14, 14, 14, 14],
 [16, 12, 13, 11],
 [10, 18, 14, 14]], dtype=int32)

In []: `#31st Oct 2025`

In [90]: `b[3]`

Out[90]: array([16, 12, 13, 11], dtype=int32)

In [91]: `b[-4]`

Out[91]: array([16, 10, 14, 15], dtype=int32)

In [92]: `b[1:3]`

Out[92]: array([[16, 10, 14, 15],
 [14, 14, 14, 14]], dtype=int32)

In [93]: `b[1,3]`

Out[93]: np.int32(15)

In [94]: `b[0,3]`

Out[94]: np.int32(19)

In [95]: `b[1,2]`

Out[95]: np.int32(14)

In [96]: `b[1:2]`

```
Out[96]: array([[16, 10, 14, 15]], dtype=int32)
```

```
In [97]: b
```

```
Out[97]: array([[10, 12, 13, 19],
                [16, 10, 14, 15],
                [14, 14, 14, 14],
                [16, 12, 13, 11],
                [10, 18, 14, 14]], dtype=int32)
```

```
In [98]: b[1,-1]
```

```
Out[98]: np.int32(15)
```

```
In [99]: b[1:-1]
```

```
Out[99]: array([[16, 10, 14, 15],
                [14, 14, 14, 14],
                [16, 12, 13, 11]], dtype=int32)
```

```
In [100... b[0:-2]
```

```
Out[100... array([[10, 12, 13, 19],
                [16, 10, 14, 15],
                [14, 14, 14, 14]], dtype=int32)
```

```
In [101... b[-5,3]
```

```
Out[101... np.int32(19)
```

```
In [102... b[-5,-3]
```

```
Out[102... np.int32(12)
```

```
In [ ]: #Operations:
```

```
In [117... a = np.random.randint(10,20,10)
a
```

```
Out[117... array([17, 13, 12, 11, 10, 12, 10, 13, 16, 13], dtype=int32)
```

```
In [118... id(a)
```

```
Out[118... 2333517777488
```

```
In [119... arr2 = np.random.randint(0,100,(10,10))  
arr2
```

```
Out[119... array([[76, 60, 82, 48, 11, 25, 87,  0,  5, 68],  
      [54, 49, 74, 94, 44, 34, 74,  1,  6, 34],  
      [11, 12, 99, 34, 53,  0,  8, 63, 75, 39],  
      [78,  6, 28, 28, 79, 59, 81, 94, 65, 31],  
      [49, 44, 73, 62, 68, 63, 58, 75, 53, 92],  
      [37, 65, 79, 49, 37, 55,  8, 69,  5, 32],  
      [34,  5, 58,  0, 83, 31,  3, 19, 40, 35],  
      [ 1, 64,  7, 63, 61, 18, 28, 34, 92, 29],  
      [31, 71, 24, 67, 75,  3, 91, 41, 69, 70],  
      [26, 79, 17, 95, 64, 21, 56, 48, 47, 93]], dtype=int32)
```

```
In [120... arr
```

```
Out[120... array([0, 1, 2, 3, 4, 5])
```

```
In [121... arr[:]
```

```
Out[121... array([0, 1, 2, 3, 4, 5])
```

```
In [122... arr[0:4]
```

```
Out[122... array([0, 1, 2, 3])
```

```
In [124... arr2[::-1]      #Reverse
```

```
Out[124... array([[26, 79, 17, 95, 64, 21, 56, 48, 47, 93],
        [31, 71, 24, 67, 75,  3, 91, 41, 69, 70],
        [ 1, 64,  7, 63, 61, 18, 28, 34, 92, 29],
        [34,  5, 58,  0, 83, 31,  3, 19, 40, 35],
        [37, 65, 79, 49, 37, 55,  8, 69,  5, 32],
        [49, 44, 73, 62, 68, 63, 58, 75, 53, 92],
        [78,  6, 28, 28, 79, 59, 81, 94, 65, 31],
        [11, 12, 99, 34, 53,  0,  8, 63, 75, 39],
        [54, 49, 74, 94, 44, 34, 74,  1,  6, 34],
        [76, 60, 82, 48, 11, 25, 87,  0,  5, 68]]], dtype=int32)
```

```
In [125... arr2[::-2]
```

```
Out[125... array([[26, 79, 17, 95, 64, 21, 56, 48, 47, 93],
        [ 1, 64,  7, 63, 61, 18, 28, 34, 92, 29],
        [37, 65, 79, 49, 37, 55,  8, 69,  5, 32],
        [78,  6, 28, 28, 79, 59, 81, 94, 65, 31],
        [54, 49, 74, 94, 44, 34, 74,  1,  6, 34]]], dtype=int32)
```

```
In [126... arr2[::-3]
```

```
Out[126... array([[26, 79, 17, 95, 64, 21, 56, 48, 47, 93],
        [34,  5, 58,  0, 83, 31,  3, 19, 40, 35],
        [78,  6, 28, 28, 79, 59, 81, 94, 65, 31],
        [76, 60, 82, 48, 11, 25, 87,  0,  5, 68]]], dtype=int32)
```

```
In [128... arr2
```

```
Out[128... array([[76, 60, 82, 48, 11, 25, 87,  0,  5, 68],
        [54, 49, 74, 94, 44, 34, 74,  1,  6, 34],
        [11, 12, 99, 34, 53,  0,  8, 63, 75, 39],
        [78,  6, 28, 28, 79, 59, 81, 94, 65, 31],
        [49, 44, 73, 62, 68, 63, 58, 75, 53, 92],
        [37, 65, 79, 49, 37, 55,  8, 69,  5, 32],
        [34,  5, 58,  0, 83, 31,  3, 19, 40, 35],
        [ 1, 64,  7, 63, 61, 18, 28, 34, 92, 29],
        [31, 71, 24, 67, 75,  3, 91, 41, 69, 70],
        [26, 79, 17, 95, 64, 21, 56, 48, 47, 93]]], dtype=int32)
```

```
In [127... arr2[:-3]
```



```
Out[127...] array([[76, 60, 82, 48, 11, 25, 87,  0,  5, 68],
        [54, 49, 74, 94, 44, 34, 74,  1,  6, 34],
        [11, 12, 99, 34, 53,  0,  8, 63, 75, 39],
        [78,  6, 28, 28, 79, 59, 81, 94, 65, 31],
        [49, 44, 73, 62, 68, 63, 58, 75, 53, 92],
        [37, 65, 79, 49, 37, 55,  8, 69,  5, 32],
        [34,  5, 58,  0, 83, 31,  3, 19, 40, 35]], dtype=int32)
```

```
In [ ]: #Step Slicing and advanced slicing in array:
```

```
In [130...] arr2[0:10:3]
```

```
Out[130...] array([[76, 60, 82, 48, 11, 25, 87,  0,  5, 68],
        [78,  6, 28, 28, 79, 59, 81, 94, 65, 31],
        [34,  5, 58,  0, 83, 31,  3, 19, 40, 35],
        [26, 79, 17, 95, 64, 21, 56, 48, 47, 93]], dtype=int32)
```

```
In [131...] arr
```

```
Out[131...] array([0, 1, 2, 3, 4, 5])
```

```
In [132...] arr.max()
```

```
Out[132...] np.int64(5)
```

```
In [133...] arr.min()
```

```
Out[133...] np.int64(0)
```

```
In [134...] arr.mean()
```

```
Out[134...] np.float64(2.5)
```

```
In [135...] arr.median()
```

```
-----  
AttributeError                                Traceback (most recent call last)  
Cell In[135], line 1  
----> 1 arr.median()  
  
AttributeError: 'numpy.ndarray' object has no attribute 'median'
```

```
In [136... from numpy import *  
a = array([1,2,3,4,9])  
median(a)
```

```
Out[136... np.float64(3.0)
```

```
In [138... b = array([1,2,3,10,9,8])  
median(b)
```

```
Out[138... np.float64(5.5)
```

```
In [ ]: #Without work on import, find median and mode
```

```
In [139... arr
```

```
Out[139... array([0, 1, 2, 3, 4, 5])
```

```
In [140... arr.reshape(3,2)
```

```
Out[140... array([[0, 1],  
                [2, 3],  
                [4, 5]])
```

```
In [141... arr.reshape(6,1)
```

```
Out[141... array([[0],  
                [1],  
                [2],  
                [3],  
                [4],  
                [5]])
```

```
In [143... arr.reshape(1,6)
```

```
Out[143... array([[0, 1, 2, 3, 4, 5]])
```

```
In [145... arr.reshape(2,3,order='C')
```

```
Out[145... array([[0, 1, 2],  
          [3, 4, 5]])
```

```
In [146... arr.reshape(2,3,order='F')      # print elements with fortran
```

```
Out[146... array([[0, 2, 4],  
          [1, 3, 5]])
```

```
In [147... arr.reshape(2,3,order='A')      # A almost give you c type output
```

```
Out[147... array([[0, 1, 2],  
          [3, 4, 5]])
```

```
In [ ]: #Indexing:
```

```
In [149... mat = np.arange(0,100).reshape(10,10)  
mat
```

```
Out[149... array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],  
          [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],  
          [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],  
          [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],  
          [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],  
          [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],  
          [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],  
          [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],  
          [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],  
          [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

```
In [150... rows = 4  
cols = 6  
mat[4,6]
```

```
Out[150... np.int64(46)
```

```
In [ ]: #Parse columns in array:
```

```
In [151... mat[:,cols]
```

```
Out[151... array([ 6, 16, 26, 36, 46, 56, 66, 76, 86, 96])
```

```
In [152... mat[:,1]
```

```
Out[152... array([ 9, 19, 29, 39, 49, 59, 69, 79, 89, 99])
```

```
In [153... mat[:,2]
```

```
Out[153... array([ 8, 18, 28, 38, 48, 58, 68, 78, 88, 98])
```

```
In [154... mat[rows,:]      #This is same as mat[rows]
```

```
Out[154... array([40, 41, 42, 43, 44, 45, 46, 47, 48, 49])
```

```
In [155... mat[rows]
```

```
Out[155... array([40, 41, 42, 43, 44, 45, 46, 47, 48, 49])
```

```
In [156... mat[:8]
```

```
Out[156... array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
                  [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
                  [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
                  [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
                  [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
                  [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
                  [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
                  [70, 71, 72, 73, 74, 75, 76, 77, 78, 79]])
```

```
In [161... mat[0:10:3]
```

```
Out[161... array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
                  [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
                  [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
                  [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

```
In [157... #Parse specific rows and cols:
mat
```

```
Out[157... array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
          [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
          [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
          [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
          [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
          [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
          [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
          [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
          [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
          [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

```
In [158... mat[2:6, 2:4]      # 1:5 --> only row part /// 1:3 -- it indicates only column parts
```

```
Out[158... array([[22, 23],
          [32, 33],
          [42, 43],
          [52, 53]])
```

```
In [159... mat[1:2, 2:4]
```

```
Out[159... array([[12, 13]])
```

```
In [160... mat[2:3, 2:3]
```

```
Out[160... array([[22]])
```

```
In [162... #Masking (Filters):
mat
```

```
Out[162... array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
          [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
          [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
          [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
          [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
          [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
          [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
          [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
          [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
          [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

```
In [163... id(mat)
```

Out[163... 2333522287120

In [164... `mat>=50`

Out[164... `array([[False, False, False, False, False, False, False, False, False, False, False],
[False, False, False, False, False, False, False, False, False, False, False],
[False, False, False, False, False, False, False, False, False, False, False],
[False, False, False, False, False, False, False, False, False, False, False],
[False, False, False, False, False, False, False, False, False, False, False],
[True, True, True, True, True, True, True, True, True, True, True],
[True, True, True, True, True, True, True, True, True, True, True],
[True, True, True, True, True, True, True, True, True, True, True],
[True, True, True, True, True, True, True, True, True, True, True],
[True, True, True, True, True, True, True, True, True, True, True]])`

In [165... `mat[mat>=50]`

Out[165... `array([50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66,
67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83,
84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])`

In [166... `mat[mat>50]`

Out[166... `array([51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67,
68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84,
85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])`

In [167... `mat[mat<=50]`

```
Out[167...] array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
        17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
        34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50])
```

```
In [168...] mat[mat==50]
```

```
Out[168...] array([50])
```

```
In [169...] mat[mat!=50]
```

```
Out[169...] array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
        17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
        34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 51,
        52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
        69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85,
        86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

```
In [ ]: #There is a python program to generate OTP in sir ka file 30th Oct2025. T11-NumPy-Crash-Course.ipynb. Check at the er
```

```
In [ ]: #Range was pending from sir ka file. Practicing here.
```

```
In [170...] range(5)
```

```
Out[170...] range(0, 5)
```

```
In [177...] r = range(0,5)
r
```

```
Out[177...] range(0, 5)
```

```
In [180...] for i in r:
            print(i)
```

```
0
1
2
3
4
```

```
In [182...] list(range(5))
```

Out[182... [0, 1, 2, 3, 4]

In [183... `range(1,10)`

Out[183... `range(1, 10)`

In [184... `list(range(1,10,3))`

Out[184... [1, 4, 7]

In [185... `y = list(range(12))`
`y`

Out[185... [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

In []: `#randn() check`

In [189... `np.random.rand(5)`

Out[189... `array([0.56943998, 0.42688489, 0.72664462, 0.42169553, 0.01998452])`

In [190... `np.random.randn(5)`

Out[190... `array([-0.50738673, -0.63114556, -2.38306086, 0.73907796, -0.46696662])`

In []: